LESSON 08 · CHEAT SHEET

Sorting with Comparators

Java 8 → Java 21 · results use the shared course dataset
course "today" = 2026-11-07

Key rules

1. `sorted()` = natural order: elements must be **Comparable**, else ClassCastException at run time. 8
2. `sorted(cmp)` works for any type. Build `cmp` from **keys**, not hand-written compare. 8
3. **One key DESC?** `comparing(key, reverseOrder())`. **Not** `.reversed()` at the end: it reverses the whole chain, nulls included.
4. Nullable key → `comparing(key, nullsLast(naturalOrder()))`.
5. Enums sort by **declaration order**; BigDecimal by value (10.0 = 10.00).
6. Sorts are **stable**; still end with a unique tie-breaker (id).
7. User text → `String.CASE_INSENSITIVE_ORDER`.

Comparator toolbox

Method	Use	Since
`comparing(key)`	sort by a Comparable key	8
`comparing(key, cmp)`	key with its own order	8
`comparingInt/Long/Double`	primitive key, no boxing	8
`thenComparing(...)`	tie-breaker	8
`reversed()`	reverse the WHOLE chain	8
`naturalOrder / reverseOrder`	Comparable types	8
`nullsFirst / nullsLast`	null-safe wrapper	8
`Map.Entry.comparingByKey/Value`	sort map entries	8
`String.CASE_INSENSITIVE_ORDER`	ignore case	1.2
`list.sort(cmp)`	in place, mutable list	8
`list.reversed()`	reverse view, no copy	21

Canonical snippets

1 · Ranking: tier DESC, total DESC, name ASC

```
comparing(CustomerView::tier, reverseOrder())
  .thenComparing(CustomerView::total, reverseOrder())
  .thenComparing(CustomerView::name);
// C003 PLATINUM 163950.25, C006 …, C010 GOLD …
```

2 · Nullable key, most recent first

```
.sorted(comparing(Activity::last,
                  nullsLast(reverseOrder()))
        .thenComparing(Activity::account))
```

3 · Map by value, top 3

```
map.entrySet().stream()
   .sorted(Map.Entry.<String, Long>comparingByValue(
             reverseOrder())
         .thenComparing(Map.Entry.comparingByKey()))
   .limit(3)  // [ACC-1003=7, ACC-1004=6, ACC-1008=6]
```

The reversed() gotcha (real output)

```
(tier, name)             C002 C005 … | C003 C006
(tier, name).reversed()  C006 C003 | … C005 C002 ×
(tier DESC, name)        C003 C006 | C001 C004 … ✓
```

reversed() also turns nullsLast into nulls **first** (Exercise08 task 3).

Mini diagram: keys in order



Real results from the course code

Call	Result
`customers.sorted()`	ClassCastException
enums `sorted()`	[SILVER, GOLD, GOLD, PLATINUM]
BigDecimal `sorted()`	[-1, 9.5, 10.00, 10.0] (stable)
`comparing(last)`, null key	NullPointerException

Call	Result
natural vs case-insensitive	[CARD, PayPal, card, paynow] / [CARD, card, paynow, PayPal]
`List.of(..).sort(..)`	UnsupportedOperationException
bottom 3 via `reversed()`	[C009, C002, C005]
stable by date	T009 stays before T010

Anti-patterns → fixes

Don't	Do
`(a, b) -> a - b`	`comparingInt` / `Integer.compare`
`.thenComparing(..).reversed()`	`comparing(k, reverseOrder())`
`sorted()` on a record	`sorted(comparing(..))`

Don't	Do
sort money as `double`	BigDecimal key
a second sort to reverse	`list.reversed()` [21]
no tie-breaker	`.thenComparing(id)`

Lesson 08 · sticky notes

Cut out, stick on your wall, and read one every morning.

sorted() needs Comparable

A record? ClassCastException at run time. Pass a Comparator.

comparing(key)

Sort by keys, not by hand-written compare().

reversed() = ALL

It flips every key and the nulls. One key DESC? reverseOrder().

Null key?

`comparing(k, nullsLast(naturalOrder()))`

Enums: declaration order

SILVER < GOLD < PLATINUM. Not alphabetical!

Map by value

```
entrySet().stream()
.sorted(comparingByValue())
```

Stable + tie-breaker

Equal keys keep input order. End with .thenComparing(id).

Java 21 reversed()

A view of the other end. No comparator, no copy.